Звіт з практичної роботи № 5

Профілювання та roofline-аналіз обчислювального ядра
Варіант 3

1. Ядро та аналітична арифметична інтенсивність

Ядро варіанта 3 - 5-точкове стенсильне оновлення явної схеми теплопровідності на 2D-сітці n×n, 500 кроків за часом: v[i][j] = u[i][j] + r·(u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1] - 4·u[i][j]), r = 0,2 (умова стійкості r ≤ 0,25). Сітки: n = 200, 300, 400, 500, 600, 800. Коректність перевірено: при r ≤ 0,25 сума тепла в області зберігається (100,000000 після 20 кроків для імпульсу 100, поки тепло не дійшло до межі).

FLOP на клітинку (за формулою): 3 додавання (сума чотирьох сусідів) + 1 множення (4·u) + 1 віднімання + 1 множення (r·) + 1 додавання (u + ...) = 7 FLOP.

Мінімальний трафік пам'яті на клітинку за крок: прочитати u[i][j] (8 байт) і записати v[i][j] (8 байт) = 16 байт; чотири сусіди беруться з кешу, бо вже завантажені для сусідніх клітинок (три рядки сітки вміщуються в кеш). Звідси I = 7/16 = 0,4375 FLOP/байт. Враховано застереження методички: рахується мінімально необхідний трафік, а не сума всіх звернень у коді (5 читань + 1 запис = 48 байт дало б хибне I = 0,146).

2. Апаратна конфігурація та еталонні бенчмарки

Процесор Intel Xeon 2,10 ГГц (2 vCPU), 8 ГБ ОЗП, Linux, V8 (Node.js v22), один потік. P_peak і B_mem виміряно окремими бенчмарками, а не взято з паспорта.

- P_peak: тісний цикл із 8 незалежними ланцюжками a = a·x + y (по 2 FLOP), $2 \cdot 10^7$ ітерацій, дані в регістрах.
- B_mem: STREAM triad c[i] = a[i] + s·b[i] на трьох масивах по $4 \cdot 10^6$ double (по 32 МБ, значно більше за кеш), 24 байти на ітерацію.

```
P_peak = 5.42 ГФлоп/с  (медіана 0.0590 с, min–max 0.0571–0.0827)
B_mem  = 10.36 ГБ/с     (медіана 0.0093 с, min–max 0.0092–0.0097)
Точка перетину I* = P_peak/B_mem = 0.523 FLOP/байт
Інтенсивність ядра I = 7/16 = 0.4375 FLOP/байт
Roofline-межа для I: min(P_peak, B_mem·I) = 4.53 ГФлоп/с -> зона, обмежена ПАМ'ЯТТЮ
```

3. Вимірювання ядра

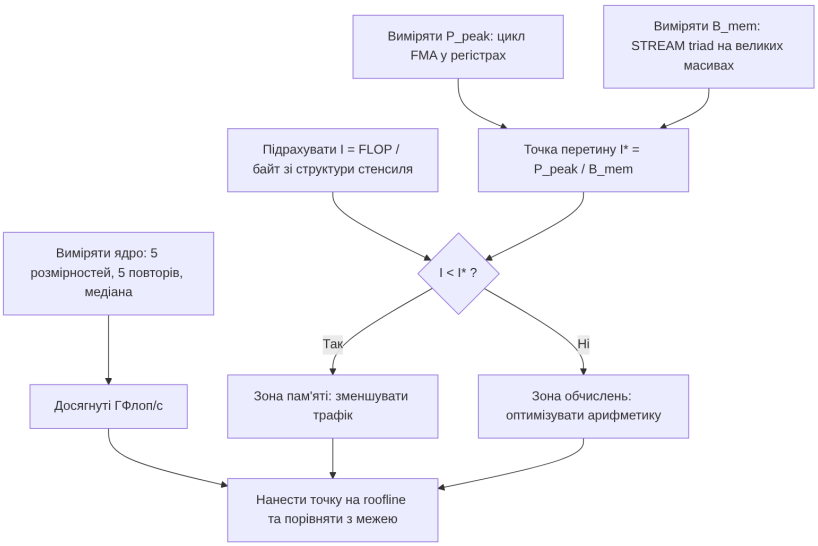


Рис. 1. Порядок roofline-аналізу (Mermaid)

Протокол: 5 повторів на кожну розмірність, перший (прогрів) відкинуто, береться медіана решти чотирьох; розкид - мінімум-максимум та міжквартильний розмах (IQR). Кількість FLOP = $(n-2)^2 \cdot 500 \cdot 7$. Режиму Release в JS немає: JIT оптимізує автоматично.

n	Медіана, с	min-max, с	IQR, с	ГФлоп/с	% від межі roofline
200	0.0675	0.0609–0.0776	0.0062	2.033	44.9 %

n	Медіана, с	min-max, с	IQR, с	ГФлоп/с	% від межі roofline
300	0.1634	0.1303–0.1739	0.0135	1.902	42.0 %
400	0.2772	0.2354–0.3706	0.0416	2.000	44.1 %
500	0.4421	0.3759–0.5038	0.1145	1.963	43.3 %
600	0.5646	0.5433–0.5757	0.0119	2.217	48.9 %
800	1.0200	0.9957–1.0432	0.0461	2.185	48.2 %

4. Roofline-діаграма

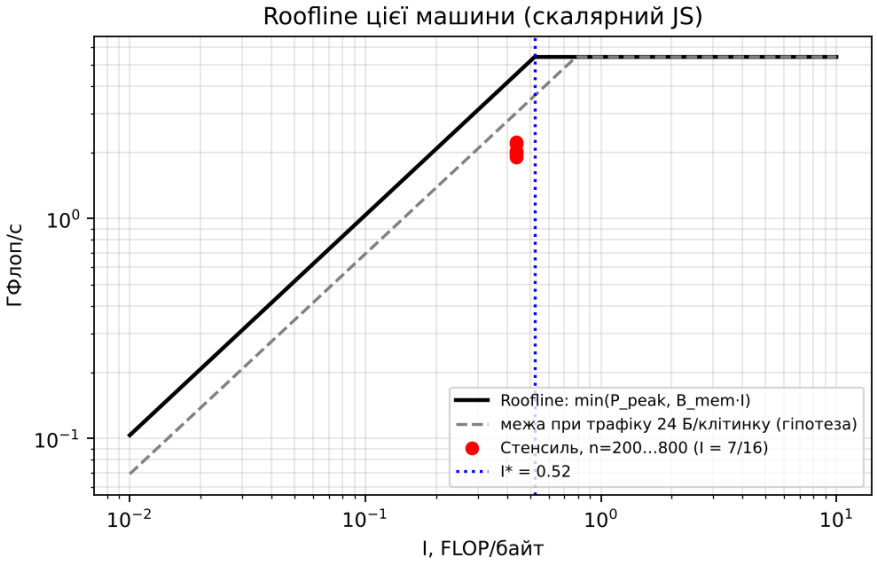


Рис. 2. Roofline: суцільна лінія - межа min(P_peak, B_mem·I); червоні точки - ядро для різних n (I однакове, бо це властивість алгоритму)

5. Висновок про зону обмеження та напрям оптимізації

Точка перетину I* = P_peak / B_mem = 5.42 / 10.36 = 0.52 FLOP/байт, інтенсивність ядра I = 0,4375 < I*, тобто ядро в зоні, обмеженій пам'яттю, але лише трохи лівіше від «лікоті» (I/I* = 0.84). Roofline-межа для нього B_mem·I = 4.53 ГФлоп/с; досягнуто 1.90-2.22 ГФлоп/с, тобто 42-49 % межі. Розрив можна частково пояснити тим, що при записі у v система спочатку читає лінію кешу (write-allocate): тоді реальний трафік ≈ 24 байти на клітинку, I ≈ 0,29, межа знижується до ≈ 3.02 ГФлоп/с, і досягнуті 2.22 ГФлоп/с становлять ≈ 73 %. Це гіпотеза, окремо не перевірялась; крім того, у JS додаються перевірки меж масивів.

Напрям оптимізації, що випливає з позиції на roofline: зменшувати трафік пам'яті, а не арифметику. Розгортання циклу чи векторизація піднімають лише дах P_peak, до якого ядро не дотягується, тож швидкодію не збільшать. Ефективніше об'єднати кілька кроків за часом в один прохід по плитках сітки (часове блокування): трафік на клітинку-крок падає приблизно в k разів, а I зростає приблизно в k разів (при k = 2 I ≈ 0,875 > I*). Лише після цього ядро перейде в зону обмеження обчисленнями, і тоді має сенс оптимізувати арифметику (наприклад, u·(1-4r) + r·(сума сусідів): 6 FLOP замість 7).

6. Відповіді на контрольні запитання

№	Відповідь
1	Арифметична інтенсивність - кількість FLOP на байт переданих даних. Це властивість алгоритму та формату даних (рахується зі структури: тут 7/16), тому не залежить від обладнання й не вимірюється секундоміром, який дає лише час.
2	P_peak і B_mem - характеристики обладнання, а не ядра: однакові для будь-якого ядра на цій машині, тож вимірюються один раз і використовуються повторно.
3	Якщо швидкодія значно нижча за min(P_peak, B_mem·I): (а) реальний трафік більший за мінімальний (write-allocate або кеш-промахи, коли рядки сітки не вміщуються в кеш); (б) JS-накладні витрати, не враховані в 7 FLOP (перевірки меж масивів, непрямі звернення). Лише після цього варто підозрювати помилку в постановці roofline.

№	Відповідь
4	При I = 0,15 FLOP/байт і I* ≈ 0,5 ядро жорстко обмежене пам'яттю. Розгортання циклу прискорює арифметику, яка й так не вузьке місце, тож сенсу не має; потрібно зменшувати обсяг переданих даних (tiling, злиття проходів, компактніший формат).
5	«Байти, виділені під масив» - обсяг пам'яті структури (тут 2 масиви по $8n^2$ байт), а «байти, передані за прохід» - реальний мінімальний трафік (16 байт на клітинку). Приклад розбіжності: сусідні елементи стенсиля читаються по 4 рази на клітинку, але з кешу, тож у трафік пам'яті входять один раз: у коді 6 звернень, а трафіку 16 байт, а не 48.
6	Запуск: вставити lab5.js в онлайн-компілятор JavaScript і натиснути Run. Якщо компілятор обриває виконання за часом, у файлі змінити const QUICK = false на true (менші сітки й 100 кроків).

Додаток. Повний код lab5.js

```
const now = (typeof performance !== "undefined" && performance.now)
  ? () => performance.now() : () => Date.now();

function median(a) {
  const s = a.slice().sort((x, y) => x - y), m = s.length >> 1;
  return s.length % 2 ? s[m] : (s[m - 1] + s[m]) / 2;
}
function quantile(sorted, q) {
  const p = (sorted.length - 1) * q, lo = Math.floor(p), hi = Math.ceil(p);
  return sorted[lo] + (sorted[hi] - sorted[lo]) * (p - lo);
}
function bench(action, repeats = 5) {
  const t = [];
  for (let r = 0; r < repeats; r++) {
    const t0 = now(); action(); const dt = (now() - t0) / 1000;
    if (r > 0) t.push(dt);
  }
  const s = t.slice().sort((a, b) => a - b);
  return { med: median(t), min: s[0], max: s[s.length - 1], iqr: quantile(s, 0.75) - quantile(s, 0.25) };
}

function peakKernel(iters) {
  let a0 = 1, a1 = 1.1, a2 = 1.2, a3 = 1.3, a4 = 1.4, a5 = 1.5, a6 = 1.6, a7 = 1.7;
  const x = 0.9999999, y = 1e-9;
  for (let i = 0; i < iters; i++) {
    a0 = a0 * x + y; a1 = a1 * x + y; a2 = a2 * x + y; a3 = a3 * x + y;
    a4 = a4 * x + y; a5 = a5 * x + y; a6 = a6 * x + y; a7 = a7 * x + y;
  }
  return a0 + a1 + a2 + a3 + a4 + a5 + a6 + a7;
}
const PEAK_ITERS = 20000000;
let sink = 0;
const tPeak = bench(() => { sink += peakKernel(PEAK_ITERS); }, 6);
const Ppeak = PEAK_ITERS * 8 * 2 / tPeak.med / 1e9;

const MEM_N = 4000000;
const ma = new Float64Array(MEM_N).fill(1), mb = new Float64Array(MEM_N).fill(2), mc = new Float64Array(MEM_N);
function triad(s) { for (let i = 0; i < MEM_N; i++) mc[i] = ma[i] + s * mb[i]; }
const tMem = bench(() => triad(3.0), 8);
const Bmem = (MEM_N * 3 * 8) / tMem.med / 1e9;

function heatSteps(n, steps) {
  let u = new Float64Array(n * n), v = new Float64Array(n * n);
  u[(n >> 1) * n + (n >> 1)] = 100;
  const r = 0.2;
  for (let s = 0; s < steps; s++) {
    for (let i = 1; i < n - 1; i++) {
      const row = i * n;
      for (let j = 1; j < n - 1; j++) {
        const k = row + j, uc = u[k];
        v[k] = uc + r * (u[k - n] + u[k + n] + u[k - 1] + u[k + 1] - 4 * uc);
      }
    }
    const t = u; u = v; v = t;
  }
  return u;
}

{
  const n = 41, u = heatSteps(n, 20);
  let sum = 0; for (let i = 0; i < u.length; i++) sum += u[i];
  console.log(`Перевірка: сума тепла після 20 кроків = ${sum.toFixed(6)} (очікується 100)`);
}

const FLOP_PER_CELL = 7, BYTES_PER_CELL = 16;
const I = FLOP_PER_CELL / BYTES_PER_CELL;
const Istar = Ppeak / Bmem;
const roof = Math.min(Ppeak, Bmem * I);

console.log("\n=== Еталонні константи цієї машини ===");

console.log(`P_peak = ${Ppeak.toFixed(2)} ГФлоп/с  (медіана ${tPeak.med.toFixed(4)} с, min–max
    ${tPeak.min.toFixed(4)}–${tPeak.max.toFixed(4)})`);
console.log(`B_mem  = ${Bmem.toFixed(2)} ГБ/с     (медіана ${tMem.med.toFixed(4)} с, min–max
    ${tMem.min.toFixed(4)}–${tMem.max.toFixed(4)})`);
console.log(`Точка перетину I* = P_peak/B_mem = ${Istar.toFixed(3)} FLOP/байт`);
console.log(`\nІнтенсивність ядра I = ${FLOP_PER_CELL}/${BYTES_PER_CELL} = ${I.toFixed(4)} FLOP/байт`);
console.log(`Roofline-межа для I: min(P_peak, B_mem·I) = ${roof.toFixed(2)} ГФлоп/с -> ` +
  (I < Istar ? "зона, обмежена ПАМ'ЯТТЮ" : "зона, обмежена ОБЧИСЛЕННЯМИ"));

const QUICK = false;
const STEPS = QUICK ? 100 : 500, sizes = QUICK ? [100, 150, 200, 250, 300] : [200, 300, 400, 500, 600, 800];
console.log(`\n=== Стенсиль, ${STEPS} кроків, повтори 5 (перший відкинуто) ===`);
console.log("n".padEnd(6) + "медіана,c".padEnd(12) + "min–max,c".padEnd(20) + "IQR,c".padEnd(10) +
    "ГФлоп/с".padEnd(10) + "% межі");
for (const n of sizes) {
  const t = bench(() => { sink += heatSteps(n, STEPS)[n + 1]; }, 5);
  const cells = (n - 2) * (n - 2) * STEPS;
```

```
  const gf = cells * FLOP_PER_CELL / t.med / 1e9;
  console.log(String(n).padEnd(6) + t.med.toFixed(4).padEnd(12) +
    `${t.min.toFixed(4)}–${t.max.toFixed(4)}`.padEnd(20) + t.iqr.toFixed(4).padEnd(10) +
    gf.toFixed(3).padEnd(10) + (100 * gf / roof).toFixed(1) + " %");
}
if (sink === 42) console.log("");
```